

SPEC.md — Hope For The Poor · Donation & Prayer Platform

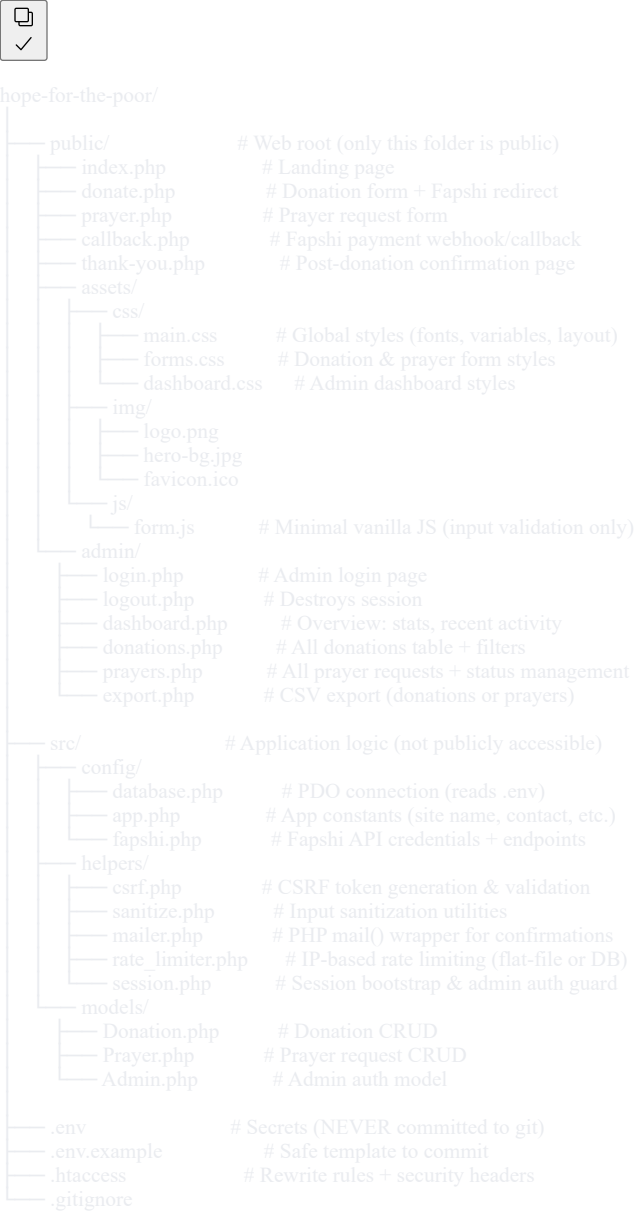
Architect: Software Architecture Document
Stack: Pure PHP 8.1+ · MySQL 8+ · Vanilla CSS · Fapshi Payment API
Approach: Security-first · No heavy frameworks · No JS libraries · Progressively enhanced

1. Project Overview

A lightweight, devotion-inspired web platform for the ministry **Hope For The Poor** that:

- Accepts one-time financial donations of any amount (minimum 100 XAF) — no account required, fully anonymous allowed
- Collects prayer requests with mandatory name + email
- Provides a secure single-admin dashboard to view donations, prayer requests, and summary statistics
- Sends automatic email confirmations to donors/prayer submitters who provide an email

2. Directory Structure



```
--
-- Table: donations
--
CREATE TABLE donations (
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  reference   VARCHAR(64) NOT NULL UNIQUE,    -- Fapshi transaction ref
  amount      INT UNSIGNED NOT NULL,          -- in XAF, min 100
  currency    CHAR(3)   NOT NULL DEFAULT 'XAF',
  donor_name  VARCHAR(120) DEFAULT NULL,      -- nullable = anonymous
  donor_email VARCHAR(254) DEFAULT NULL,      -- nullable = anonymous
  donor_phone VARCHAR(20)  DEFAULT NULL,      -- nullable
  status      ENUM('pending','completed','failed','cancelled')
              NOT NULL DEFAULT 'pending',
  fapshi_payload JSON DEFAULT NULL,           -- raw webhook payload
  ip_address  VARBINARY(16) NOT NULL,         -- stored as INET6_ATON()
  created_at  DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_status  (status),
  INDEX idx_created (created_at),
  INDEX idx_reference (reference)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
--
-- Table: prayer_requests
--
CREATE TABLE prayer_requests (
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(120) NOT NULL,
  email       VARCHAR(254) NOT NULL,
  subject     VARCHAR(200) DEFAULT NULL,
  message     TEXT          NOT NULL,
  is_read     TINYINT(1)   NOT NULL DEFAULT 0,
  is_prayed   TINYINT(1)   NOT NULL DEFAULT 0, -- admin marks as prayed
  ip_address  VARBINARY(16) NOT NULL,
  created_at  DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_is_read (is_read),
  INDEX idx_created (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
--
-- Table: admin_users
--
CREATE TABLE admin_users (
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  username    VARCHAR(60) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,        -- password_hash() bcrypt
  last_login  DATETIME  DEFAULT NULL,
  created_at  DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
--
-- Table: rate_limit (IP-based brute-force protection)
--
CREATE TABLE rate_limit (
  ip_address  VARBINARY(16) NOT NULL,
  action      VARCHAR(40) NOT NULL,           -- 'donate', 'prayer', 'login'
  attempt_count SMALLINT UNSIGNED NOT NULL DEFAULT 1,
  window_start DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (ip_address, action)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

4. Application Pages

4.1 Landing Page (public/index.php)

Sections (top to bottom):

Section	Content
Header / Nav	Logo · Ministry name · Nav links: Home, Donate, Prayer Request, Contact
Hero	Full-width devotion-inspired banner image · headline · sub-headline · two CTA buttons: "Give Now" and "Send Prayer"
About	Short paragraph about the ministry's mission
How to Donate	3-step visual (Choose amount → Pay securely → Receive confirmation)
Prayer Banner	Warm call-to-action: "We pray for you. Share your request."
Contact Section	Email: achirihilary44@gmail.com · WhatsApp/SMS: +237 654 045 897 and +237 680 828 762 · Social icons (Facebook, Instagram, Twitter/X, YouTube — links provided by admin in .env)
Footer	Copyright · "Powered with Faith" · Privacy note

4.2 Donation Form (public/donate.php)

Fields:

- Amount (number input, min=100, step=100, placeholder="Enter amount in XAF")
- Name (*optional*) — with a visible "Donate Anonymously" checkbox that clears/disables name + email fields
- Email (*optional, shown only if not anonymous*)
- Phone (*optional — used by Fapshi for Mobile Money prompt*)
- CSRF hidden token
- Submit button: "Give Now"

Flow:

1. Server-side validation (amount $\geq$ 100, email format if provided)
2. Insert donations record with status = 'pending'
3. Call Fapshi API to initiate payment → receive payment URL
4. Redirect user to Fapshi hosted payment page
5. Fapshi calls back callback.php (webhook) when payment resolves

4.3 Prayer Request Form (public/prayer.php)

Fields:

- Name (*required*)
- Email (*required*)
- Subject (*optional*)
- Prayer message (*required, textarea, max 1000 chars*)
- CSRF hidden token
- Submit button: "Send My Prayer"

Flow:

1. Validate all required fields server-side
2. Rate-limit: max 3 submissions per IP per hour
3. Insert into prayer_requests
4. Send confirmation email to submitter via mailer.php
5. Redirect to confirmation message on same page (PRG pattern)

4.4 Fapshi Callback (public/callback.php)

- Accepts POST from Fapshi with transaction reference + status
- Validates the request using Fapshi's API secret (HMAC or token header — match Fapshi docs)
- Looks up donations.reference and updates status accordingly
- On completed: send thank-you email to donor (if email was provided)
- Responds with HTTP 200 immediately — no HTML output

4.5 Thank You Page (public/thank-you.php)

- Shown after a completed payment redirect (Fapshi return URL)
- Reads ?ref= from query string, verifies it exists and belongs to a completed donation
- Shows personalized message with amount and a blessing
- Links back to home

5. Admin Dashboard

Access Control

- Single admin user, created via a one-time CLI seed script (scripts/seed_admin.php)
- Session-based auth: \$_SESSION['admin_id'] checked on every admin page via session.php guard
- Session expires after 30 minutes of inactivity (session.gc_maxlifetime)
- Login page has rate limiting: 5 failed attempts per IP per 15 minutes → 15-minute lockout
- Passwords hashed with password_hash(\$pass, PASSWORD_BCRYPT, ['cost' => 12])

Dashboard Pages

admin/login.php

- Username + password form
- CSRF protected
- Rate-limited

admin/dashboard.php — Overview

Widgets displayed:

Widget	Data
Total Donations (all time)	SUM(amount) WHERE status='completed'
Donations This Month	SUM + COUNT for current month
Pending Transactions	COUNT WHERE status='pending'
Unread Prayer Requests	COUNT WHERE is_read=0
Recent Donations	Last 5 completed donations (table)
Recent Prayers	Last 5 unread prayer requests (table)

admin/donations.php — Donations Manager

- Filterable table: by status, by date range
- Columns: Date · Name (or "Anonymous") · Amount · Status · Reference
- Pagination: 25 per page
- CSV Export button → export.php?type=donations

admin/prayers.php — Prayer Requests Manager

- Table: Date · Name · Email · Subject · Status (Unread / Read / Prayed For)
- Click to expand full message inline
- Two action buttons per row: **Mark as Read** · **Mark as Prayed For**
- CSV Export button → export.php?type=prayers

admin/export.php

- Outputs CSV with proper headers (Content-Disposition: attachment)
- Respects current filter params passed from donations/prayers pages
- Access guarded (admin session required)

6. Fapshi Integration

API Calls (from src/config/fapshi.php + inline in donate.php / callback.php)



Base URL: https://live.fapshi.com (or sandbox for testing)

1. Initiate Payment:
POST /initiate-pay
Headers: apiuser: {FAPSHI_USER}, apikey: {FAPSHI_KEY}
Body: { amount, email (optional), redirectUrl, message }
Response: { link, transId }

2. Verify Payment (in callback.php):
GET /payment-status/{transId}
Headers: apiuser, apikey
Response: { status, amount, ... }

- Store transId as donations.reference
- **Never** trust Fapshi's webhook payload alone — always re-verify status via GET call
- Log full raw response to donations.fapshi_payload for audit trail

7. Security Architecture

Input Handling

- All \$_POST and \$_GET values run through sanitize.php:
 - htmlspecialchars() on output
 - filter_var() for email, integer validation
 - Reject unexpected fields (whitelist-only approach)
- All DB queries use **PDO prepared statements** — zero raw SQL interpolation

CSRF Protection

- Every form includes a hidden token: <input type="hidden" name="csrf_token" value="...">
- Token stored in \$_SESSION['csrf_token'], regenerated per session
- Validated server-side before any form processing; mismatch = 403

Session Security



php

```
session_start([
    'cookie_secure' => true,    // HTTPS only
    'cookie_httponly' => true,  // No JS access
    'cookie_samesite' => 'Strict',
    'use_strict_mode' => true,
]);
session_regenerate_id(true);    // Called on every privilege change
```

Rate Limiting

- Stored in rate_limit table (works across multiple server instances)
- Limits:
 - Donation submission: 10 per IP per hour
 - Prayer submission: 3 per IP per hour
 - Admin login attempts: 5 per IP per 15 minutes

HTTP Security Headers (.htaccess)


✓

apache

Header always set X-Frame-Options "DENY"

Header always set X-Content-Type-Options "nosniff"

Header always set Referrer-Policy "strict-origin-when-cross-origin"

Header always set Content-Security-Policy "default-src 'self'; style-src 'self' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com; img-src 'self' data:; connect-src 'self' https://i

Header always set Permissions-Policy "geolocation=(), microphone=(), camera=()"

Callback Endpoint Protection

- callback.php only accepts POST requests
- Validates Fapshi Authorization header token
- Re-verifies payment status via Fapshi API before updating DB
- Responds 200 regardless to prevent enumeration

Environment Variables (.env)


✓

env

DB_HOST=localhost

DB_NAME=hope_db

DB_USER=hope_usr

DB_PASS=strongpassword

FAPSHI_USER=your_api_user

FAPSHI_KEY=your_api_key

FAPSHI_SANDBOX=false

SITE_URL=https://yourdomain.com

ADMIN_EMAIL=achirihilary44@gmail.com

SOCIAL_FACEBOOK=https://facebook.com/...

SOCIAL_INSTAGRAM=https://instagram.com/...

SOCIAL_TWITTER=https://twitter.com/...

SOCIAL_YOUTUBE=https://youtube.com/...

MAIL_FROM=noreply@yourdomain.com

MAIL_FROM_NAME=Hope For The Poor

- .env is **never committed** to version control
- Parsed by a simple src/config/env.php reader — no external library needed

8. Visual Design Language

Aesthetic: Sacred, warm, human — not corporate. Inspired by illuminated manuscripts and candlelit devotion.

Color Palette:

Name	Hex	Use
Deep Claret	#5B1A2E	Primary headings, buttons
Warm Gold	#C9922A	Accents, dividers, CTA hover
Ivory Parchment	#FAF6EE	Page background
Soft Charcoal	#3D3530	Body text
Muted Sage	#7A9E7E	Prayer section accent
White	#FFFFFF	Cards, forms

Typography:

- Display / Headings: **EB Garamond** (Google Fonts, serif) — dignified, timeless
- Body: **Lato** (Google Fonts, sans-serif) — readable at small sizes
- Load from Google Fonts with `display=swap` for performance

CSS Variables (in `:root`):



css

```
:root {
  --color-primary: #5B1A2E;
  --color-accent: #C9922A;
  --color-bg: #FAF6EE;
  --color-text: #3D3530;
  --color-prayer: #7A9E7E;
  --font-display: 'EB Garamond', Georgia, serif;
  --font-body: 'Lato', Arial, sans-serif;
  --radius: 6px;
  --shadow: 0 2px 12px rgba(91,26,46,0.08);
  --max-width: 1100px;
  --transition: 0.2s ease;
}
```

Signature Element: A thin gold cross divider (✦) used between sections — simple, recognisable, devotional. No images needed for it, pure CSS + unicode.

9. Email Notifications

Using PHP's built-in `mail()` (works on standard shared hosting):

Trigger	Recipient	Content
Successful donation (with email)	Donor	"Thank you for your gift of X XAF. God bless you."
Prayer request submitted	Submitter	"We received your prayer request and are praying for you."
New prayer request	Admin email	Alert with name, subject (not full message for privacy)

For production on VPS, swap `mail()` for SMTP via a tiny `PHPMailer`-style socket — but `mail()` is the default to avoid dependencies.

10. Development Phases

Phase 1 — Foundation

- ☐ Set up directory structure and `.htaccess` rules
- ☐ Create `.env` reader and DB connection (PDO)
- ☐ Build MySQL schema and run migrations
- ☐ Implement CSRF helper, session helper, sanitize helper

Phase 2 — Public Pages

- ☐ Landing page (`index.php`) with all sections
- ☐ Donation form + Fapshi API integration
- ☐ Callback handler + payment verification
- ☐ Thank-you page
- ☐ Prayer request form
- ☐ CSS: `main.css`, `forms.css`

Phase 3 — Admin Dashboard

- ☐ Login page + session auth guard
- ☐ Dashboard overview with stats widgets
- ☐ Donations manager (table + filters + CSV export)
- ☐ Prayer request manager (read/prayed status)
- ☐ `dashboard.css`

Phase 4 — Security Hardening

- ☐ Rate limiting (all forms + login)
- ☐ HTTP security headers audit
- ☐ Input whitelist review
- ☐ Admin password seed script
- ☐ Test Fapshi sandbox end-to-end

Phase 5 — Polish & Deploy

- ☐ Email notification testing
- ☐ Mobile responsiveness audit
- ☐ Cross-browser check (Chrome, Firefox, Safari)
- ☐ Set FAPSHI_SANDBOX=false
- ☐ Point domain, configure SSL (Let's Encrypt)
- ☐ Set DB user to minimum privileges (SELECT, INSERT, UPDATE only — no DROP)

11. Hosting Requirements

- PHP 8.1+
- MySQL 8.0+ (or MariaDB 10.6+)
- HTTPS (SSL certificate required — Fapshi requires it for callbacks)
- mail() enabled, or SMTP credentials available
- .htaccess support (Apache) **or** equivalent Nginx rewrite config

12. Out of Scope (for now)

The following are deliberately excluded to keep the project simple:

- User accounts / donor login
- Recurring/scheduled donations
- Multiple admin users or roles
- SMS notifications (WhatsApp number is for manual contact only)
- Payment history per donor (since anonymous donors are supported)
- Multi-language support
- CMS / editable page content from dashboard

These can be added in a future version without breaking this architecture.

"Give, and it will be given to you." — Luke 6:38
Built with faith. Secured with care.